

Algoritmi e Strutture Dati–parte I

lez. 3

Francesco Camastra

francesco.camastra@uniparthenope.it

Docente

- Prof. Francesco Camastra

Orario lezioni

- Mercoledì 11.00-13.00
(aula 11, Secondo Piano)
- Giovedì 9.00-11.00 (aula 11, Secondo Piano)

Ricevimento

- Orario: Mercoledì 14.00-18.00
- Dove: Stanza 430, Quarto piano, Centro Direzionale, Isola C4
- Per il ricevimento è **OBBLIGATORIO** inviare una email di prenotazione almeno due giorni prima del giorno di ricevimento.
- email del Docente:
francesco.camastra@uniparthenope.it
- Pagina del Docente:
dist.uniparthenope.it/francesco.camastra

Prerequisiti

- Programmazione I
- Matematica I
- Programmazione II
- **NOTA BENE:** Le propedeuticità non sono obbligatorie ... Ossia potreste anche sostenere l'esame senza aver dato Programmazione I e Programmazione II

Frequenza

- La Frequenza del corso non è obbligatoria

Modalità dell' esame

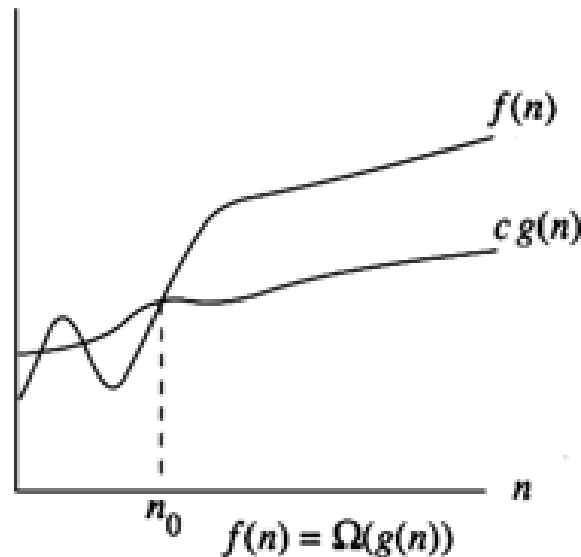
- Prova scritta di Algoritmi e Strutture Dati I ovvero Due prove intercorso (validità quattro appelli)
- Svolgimento di un progetto Algoritmi e Strutture Dati II da scegliersi tra una lista di progetti che verrà comunicata dal docente (validità quattro appelli).
- Superato lo scritto di ASD-I ed il progetto di ASD-II si accede all' Esame Orale congiunto di ASD-I e ASD-II

Testi Consigliati

- ALGORITMI:
- Cormen, Leiserson, Rivest, Stein
*Introduzione agli Algoritmi e
alle strutture Dati*
(seconda edizione o terza edizione)
Ed. Mc Graw-Hill.

La notazione Ω : Definizione

- Date $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ diremo che $f(n) = \Omega(g(n))$ se e solo se $\exists c > 0, \exists n_0$ tale che, $f(n) \geq cg(n), \forall n \geq n_0$



Legami tra O e Ω

- Tra O e Ω vale la **Regola della Dualità**, ossia vale $f(n) = \Omega(g(n))$ se e solo se $g(n) = O(f(n))$
- Per provarla basta ricordare che $f(n) = \Omega(g(n))$ implica $f(n) \leq cg(n)$, per un c opportuno e per un $n \geq n_0$.
- A sua volta $f(n) \geq cg(n) \Leftrightarrow g(n) \leq \frac{1}{c}f(n)$ per un c opportuno e per un $n \geq n_0$.
- Ma $g(n) \leq \frac{1}{c}f(n) \Leftrightarrow g(n) = O(f(n))$

Esempi

- $a^n = O(n!) \Leftrightarrow n! = \Omega(a^n)$
- $n^2 = O(n^3) \Leftrightarrow n^3 = \Omega(n^2)$
- $n = O(n \log n) \Leftrightarrow n \log n = \Omega(n)$
- $\forall a, b > 1 \quad a > b \Rightarrow a^n = \Omega(b^n)$
- $\forall a > 1, k > 0 \quad a^n = \Omega(n^k)$
- $\forall a, b > 0 \quad a > b \Rightarrow n^a = \Omega(n^b)$
- $\forall a, b, \varepsilon > 0 \quad n^{\frac{1}{\varepsilon}} = \Omega\left((\log n^b)^a\right)$

Proprietà di Ω

- $f(n) = \Omega(g(n)) \Rightarrow af(n) = \Omega(g(n)), \forall a > 0$
- $f(n) = \Omega(g(n)), h(n) = \Omega(i(n)) \Rightarrow$
 $f(n) + h(n) = \Omega(g(n) + i(n))$
- $f(n) = \Omega(g(n)), h(n) = \Omega(i(n)) \Rightarrow$
 $f(n)h(n) = \Omega(g(n)i(n))$
- $f(n) = \Omega(g(n)), g(n) = \Omega(h(n)) \Rightarrow$
 $f(n) = \Omega(h(n)).$

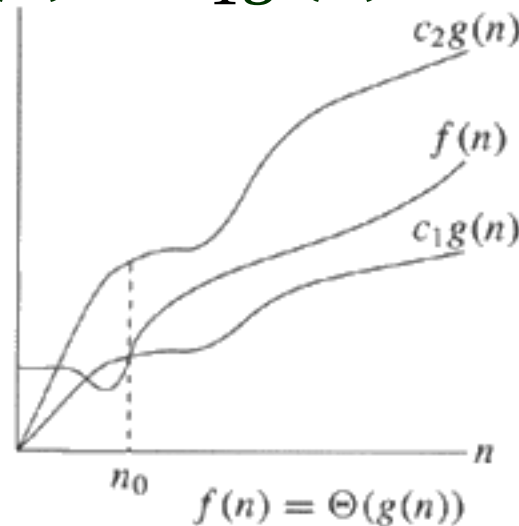
Proprietà di Ω : Esempi

- $n = \Omega(\log n) \Rightarrow \frac{1}{4}n = \Omega(\log n)$
- $n = \Omega(\log n), n^{\frac{3}{2}} = \Omega(n) \Rightarrow n^{\frac{3}{2}} + n = \Omega(n + \log n)$
- $n = \Omega(\log n), n^{\frac{3}{2}} = \Omega(n) \Rightarrow n^{\frac{5}{2}} = \Omega(n \log n)$
- $n! = \Omega(n^{900}), n^{900} = \Omega(n^{100}) \Rightarrow n! = \Omega(n^{100})$

La notazione Θ : Definizione

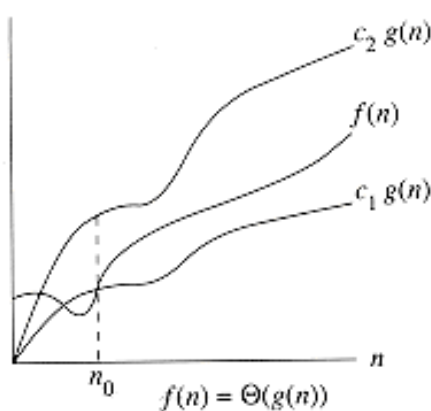
- Date $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ diremo che $f(n) = \Theta(g(n))$ se e solo se $\exists c_1, c_2 > 0, \exists n_0$ tale che,

$$c_2 g(n) \geq f(n) \geq c_1 g(n), \forall n \geq n_0$$

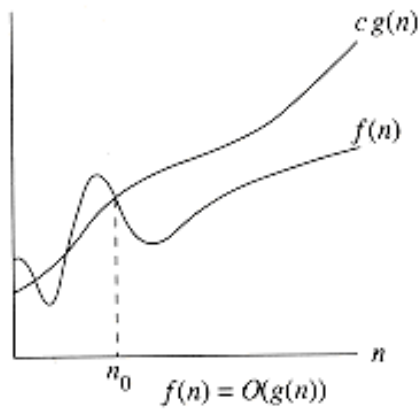


Esempi di Uso delle notazioni O , Θ , Ω

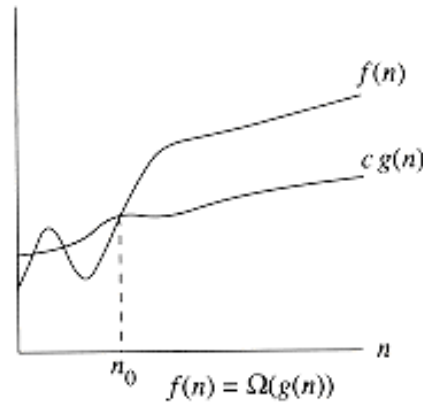
- É corretto dire che: $n^3 + n \log n + 8 = O(n^3)$
- Ma é piú preciso: $n^3 + n \log n + 8 = \Theta(n^3)$
- Allora come scegliere tra le varie notazioni ?



(a)



(b)



(c)

Scelta tra le notazioni O , Θ , Ω :

Metodo Generale

- Se $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c \neq 0 \Rightarrow f(n) = O(g(n))$ ma
anche $g(n) = O(f(n)) \Rightarrow \mathbf{f(n) = \Theta(g(n))}$
- Se $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \Rightarrow f(n) = O(g(n))$ ma
anche $g(n) \neq O(f(n)) \Rightarrow \mathbf{f(n) = O(g(n))}$
- Se $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \Rightarrow g(n) = O(f(n))$ ma
anche $f(n) \neq O(g(n)) \Rightarrow \mathbf{f(n) = \Omega(g(n))}$

Esempi

- Se $f(n) = n^3 + n \log n + 8$, $g(n) = n^3 + 7$ allora
$$\lim_{n \rightarrow \infty} \frac{n^3 + n \log n + 8}{n^3 + 7} = 1 \neq 0 \Rightarrow f(n) = \Theta(g(n))$$
- Se $f(n) = \log n + 8$, $g(n) = 2n + 7$ allora
$$\lim_{n \rightarrow \infty} \frac{\log n + 8}{2n + 7} = 0 \Rightarrow f(n) = o(g(n))$$

Prova del metodo generale

- Per semplicità proviamo la prima relazione. Supponiamo che sia $\lim_{n \rightarrow \infty} \frac{f(x)}{g(x)} = p \neq 0$ e che inoltre prendiamo un δ e c tali che $\delta = p$ e $c = 2p$. Per la definizione di limite abbiamo che $\left\| \frac{f(x)}{g(x)} - p \right\| \leq \delta$.
- Questo implica che $\left\| \frac{f(x)}{g(x)} \right\| \leq \delta + p$. Poiché $\delta + p = c$ allora $\left\| \frac{f(x)}{g(x)} \right\| \leq c \Rightarrow f(n) \leq c g(n)$.

(cont.)

- Quindi abbiamo $f(n) \leq cg(n)$ per un $n \geq n_0$ ossia $f(n) = O(g(n))$.
- E $g(n) = O(f(n))$ é una conseguenza banale in quanto $\lim_{n \rightarrow \infty} \frac{f(x)}{g(x)} = p \implies \lim_{n \rightarrow \infty} \frac{g(x)}{f(x)} = \frac{1}{p}$ e quindi si applica il precedente ragionamento scambiando semplicemente i ruoli tra $f(n)$ e $g(n)$.

Esercizi

- Stabilire se sono vere o false le seguenti affermazioni:
 - $n^5 - n^3 + 1 = O(n^5)$?
 - $n^5 - n^3 + 1 = O(n^2)$?
 - $n^5 - n^3 + 1 = O(n^7)$?
 - $n^5 - n^3 + 1 = \Omega(n^5)$?
 - $n^5 - n^3 + 1 = \Omega(n^2)$?
 - $n^5 - n^3 + 1 = \Omega(n^7)$?
 - $n^5 - n^3 + 1 = \Theta(n^5)$?
 - $n^5 - n^3 + 1 = \Theta(n^2)$?
 - $n^5 - n^3 + 1 = \Theta(n^7)$?

Esercizi

- Per le seguenti funzioni $f(n)$ e $g(n)$ stabilire se vale $f(n) = O(g(n))$ o se vale $g(n) = O(f(n))$:
 - $f(n) = n^2 - n$, $g(n) = 8n$
 - $f(n) = n + 2\log n$, $g(n) = 4n(\log n)^2$
 - $f(n) = n \log n$, $g(n) = 3n\sqrt{n}$
 - $f(n) = n^2 + 50n$, $g(n) = \frac{1}{100}n^3$
 - $f(n) = (\log n)^2$, $g(n) = n^2 - 49n$
 - $f(n) = 10n \log n + 3n$, $g(n) = 3n\sqrt{n}$

Analisi di un algoritmo

- Per **Analisi di un algoritmo** si intende la valutazione delle risorse richieste da un algoritmo per risolvere un problema dato.
- Per risorse intendiamo essenzialmente:
 - **tempo** (o tempo di esecuzione) necessario all'algoritmo per produrre l' output richiesto;
 - **memoria** utilizzata dall' algoritmo durante la sua esecuzione.
- Analizzeremo gli algoritmi in base al loro tempo di esecuzione.

Schema d' Implementazione di un algoritmo

- Il **Programmatore** scrive il programma implementando l' algoritmo;
- Il programma viene compilato e viene generato il file eseguibile;
- Il file è eseguito sul calcolatore.

Tempo d' Esecuzione

- Il tempo di esecuzione dipende:
 - dall' abilità del programmatore nella stesura del programma e dal linguaggio di programmazione utilizzato;
 - dal linguaggio macchina del computer;
 - dall' eseguibile generato dal computer;
 - dalla dimensione dell' input (o taglia del problema);
 - da come é fatto l' input (array)
 - dal numero di operazioni elementari eseguite.

Tempo d' Esecuzione (cont.)

- Useremo le notazioni asintotiche O , Θ , Ω per misurare il tempo d' esecuzione, ossia misureremo il tempo a meno di una costante moltiplicativa.
- Non ci interessano le costanti moltiplicative in quanto dipendono da fattori difficilmente quantificabili (ad es. abilità del programmatore, linguaggio macchina, eseguibile)
- Per dimensioni di input molto grandi, le costanti moltiplicative influenzano in modo irrilevante il tempo d' esecuzione.

Operazioni elementari

- Misureremo il tempo d' esecuzione (o complessità dell' algoritmo in termini del numero delle operazioni elementari utilizzate. Esempi di operazione elementare sono:
 - Addizione e moltiplicazione di numeri (piccoli)
 - confronto tra numeri
 - spostamento di un puntatore
- Assumiamo che ogni operazione elementare richieda un **tempo costante d'esecuzione**, che rappresenta la **nostra unità elementare di tempo**.

La dimensione dell' input

- Il tempo d' esecuzione dipende dalla dimensione (o taglia) dell' input. Per taglia di input si intende:
 - il numero dei bits necessari a rappresentarlo (**criterio di costo logaritmico**);
 - il numero degli elementi che lo costituiscono (**criterio di costo uniforme**).
 - Tali criteri coincidono a meno di un fattore costante moltiplicativo, nell' ipotesi che ogni istruzione del computer richieda un' unità elementare di tempo e che ogni elemento richieda una parola di memoria.

Forma dell' input

- La forma dell' input può influenzare la complessità dell' algoritmo. Ad esempio, ordinare n elementi di un array è più veloce se l' array è quasi ordinato.
- Nell' analisi solitamente si analizzerà:
 - il **caso migliore**, ossia la forma più favorevole dell' input;
 - il **caso peggiore**, ossia la forma più sfavorevole dell' input.
 - il **caso medio**.

Forma dell' input (cont.)

- La complessità di un algoritmo A su un input di dimensione n verrà indicato con $T(n)$. $T(n)$ indica il massimo del tempo richiesto su tutti i possibili input di dimensione n .
- Se $T(n)$ é $O(n^2)$ significa che l' algoritmo A richiede un tempo non superiore a cn^2 . Mentre se $T(n)$ é $\Omega(n^2)$ richiede un tempo almeno pari a cn^2 .

Calcolo delle complessità di tempo

- Utilizzeremo le seguenti regole:
 - Il **costo di istruzioni semplici** (ad es., assegnazione, lettura/scrittura di variabili) é $O(1)$.
 - Il costo di istruzioni del tipo **if then else** é pari al tempo per effettuare il confronto, solitamente $O(1)$ piú **il costo dell' alternativa piú costosa**.
 - il costo dei loop (**do, for, repeat, while**) é pari alla **somma del costo di ogni singola iterazione**.

Insertion Sort

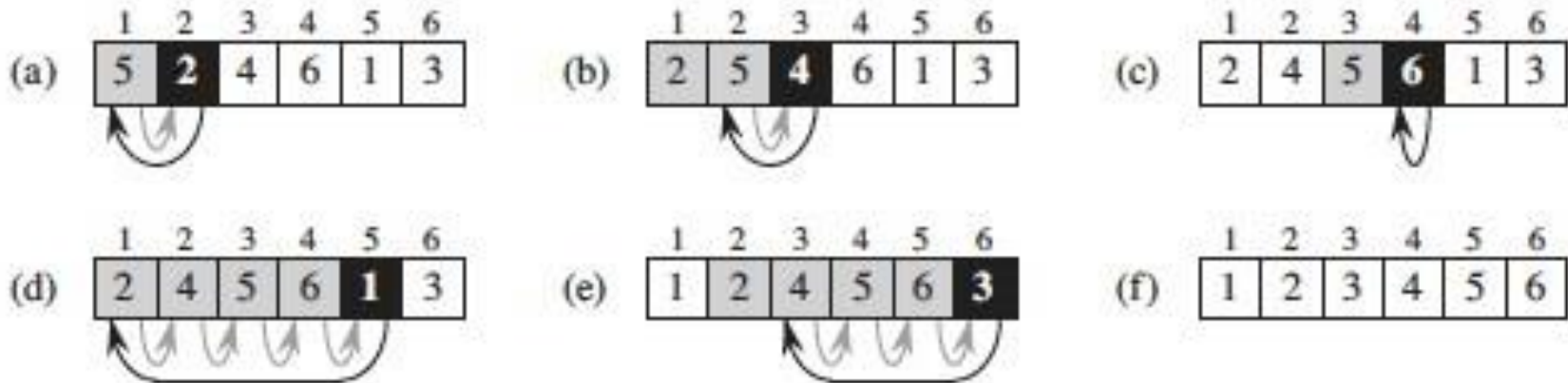
- L' Ordinamento e` un' operazione fondamentale in Informatica e pertanto sono stati sviluppati un gran numero di algoritmi di ordinamento.
- L' Insertion Sort e` l' algoritmo di ordinamento piu` semplice. E` un algoritmo efficiente quando si deve ordinare un piccolo numero di elementi
- L' algoritmo riproduce l' operazione che fa un giocatore di bridge (o di ramino) quando ordina le proprie carte. Si inizia con al mano sinistra vuota e si prende una carta, alla volta, dal tavolo inserendola nella giusta posizione. Per trovare la giusta posizione si confronta la carta con ogni carta presente nella mano sinistra.

Insertion-Sort (A)

1. For $j \leftarrow 2$ to $A.length$
2. $key \leftarrow A[j]$
3. // Inserisce $A[j]$ nella sequenza ordinata $A[1 \dots j - 1]$
4. $i \leftarrow j - 1$
5. While ($i > 0$ and $A[i] > key$)
6. $A[i+1] \leftarrow A[i]$
7. $i \leftarrow i - 1$
8. $A[i+1] \leftarrow key$

Ogni elemento dal secondo all' ultimo viene estratto dal mazzo e confrontato con tutti i precedenti che slittano in avanti se sono più grandi.
Al termine l'elemento entra nella locazione lasciata libera.

Insertion Sort: Esempio di Funzionamento



INSERTION SORT (2)

- L'indice j indica l'elemento corrente (istruzione 1) e ne viene memorizzato il suo valore (istruzione 2)
- Il vettore $A[1 \dots j - 1]$ indica gli elementi già ordinati mentre il vettore $A[j \dots n]$ quelli ancora da ordinare (n è il numero degli elementi da ordinare)
- Per ogni elemento viene trovata la giusta posizione mediante il ciclo interno (istruzioni 4,5,6) del WHILE
- Alla fine l'elemento viene inserito nella posizione corretta (istruzione 7)

Insertion Sort: Invariante di Ciclo

- All' inizio di ogni iterazione del ciclo for, il sottoarray $A[1 \dots j - 1]$ è ordinato ed è formato dagli stessi elementi che originariamente erano in $A[1 \dots j - 1]$

Invariante di Ciclo

- Utilizziamo le invarianti di ciclo per aiutarci a capire perché un algoritmo è corretto. Dobbiamo dimostrare tre cose su un' invariante di ciclo:
 - **Inizializzazione**: è vera prima della prima iterazione del ciclo;
 - **Conservazione**: se è vera prima di un' iterazione del ciclo, rimane vera prima della successiva iterazione;
 - **Conclusione**: quando il ciclo termina, l' invariante fornisce un' utile proprietà che ci aiuta a dimostrare che l' algoritmo è corretto.

Invariante di Ciclo per Insertion Sort: Inizializzazione

- Dimostriamo che l' invariante di ciclo è vera prima della prima iterazione del ciclo, quando $j = 2$.
- Il sottoarray $A[1 \dots j - 1]$ è formato dal solo elemento $A[1]$, che infatti è l' elemento originale in $A[1]$.
- Inoltre, questo sottoarray è ordinato (essendo formato da un solo elemento) e ciò dimostra che l' invariante di ciclo è vera prima della prima iterazione del ciclo.

Invariante di Ciclo per Insertion Sort: Conservazione

- Il corpo del ciclo for esterno opera spostando $A[j - 1]$, $A[j - 2]$, $A[j - 3]$ e così via di una posizione verso destra, finché non troverà la posizione appropriata per $A[j]$ (righe 4-7), dove lo inserirà (riga 8).
- Il sottoarray $A[1 \dots j]$ è ordinato ed è formato dagli stessi elementi che originariamente erano in $A[1 \dots j]$. Perciò, **l'incremento di j per la successiva iterazione del ciclo for preserva l'invariante di ciclo.**

Invariante di Ciclo per Insertion Sort: Conclusione

- La condizione che determina la conclusione del ciclo for è: $j > A.length = n$. Poiché ogni iterazione del ciclo aumenta j di 1, alla fine del ciclo si avrà $j = n + 1$.
- Sostituendo j con $n + 1$ nella formulazione dell'invariante di ciclo, otteniamo che il sottoarray $A[1 \dots n]$ è formato dagli elementi ordinati che si trovavano originariamente in $A[1 \dots n]$. Ma il sottoarray $A[1 \dots n]$ è l'intero array e quindi l'array è ordinato. Pertanto **l' algoritmo è corretto.**